ECHO SERVER USING TCP

Complete Explanation & Study Guide — Networks Laboratory, Assignment 3

1. What Is a Socket?

A socket is an endpoint for sending and receiving data across a network. Think of it as a virtual "plug" — one program plugs into the network at one address (its IP + port), and another program plugs in elsewhere; once connected, they can exchange bytes as if they were reading and writing to a file. Python's built-in socket module wraps the operating system's networking API so we can create these endpoints with a few lines of code.

Every socket needs two things to identify where it lives on the network:

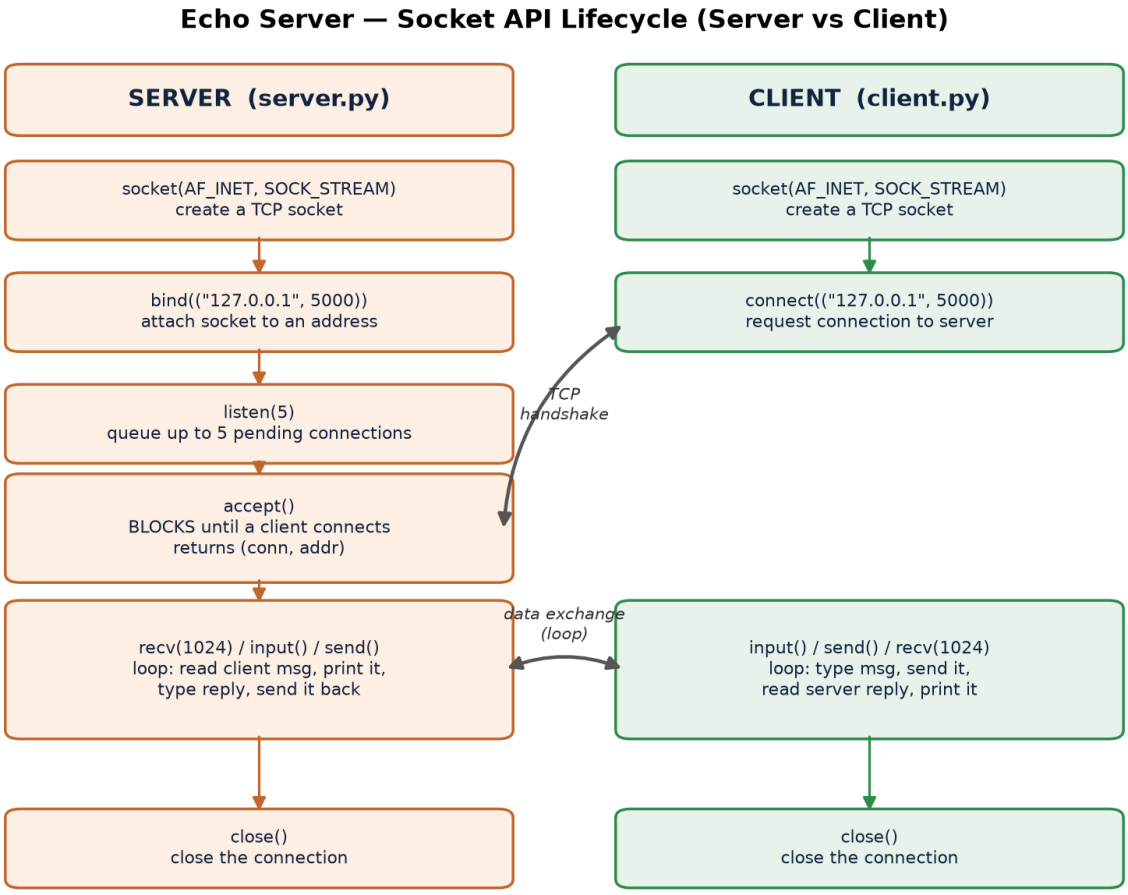
- IP address — which machine (127.0.0.1 = "this machine", called localhost)
- Port number — which application on that machine (we used 5000, an arbitrary unused port)

2. TCP vs UDP — Why TCP Here?

SOCK_STREAM in `socket.socket(AF_INET, SOCK_STREAM)` selects TCP (Transmission Control Protocol) rather than UDP. TCP is connection-oriented and reliable: before any data flows, the client and server perform a handshake to establish a connection, and TCP guarantees that bytes arrive in order and without loss (retransmitting automatically if needed). This makes it the right choice for a chat-style Echo Server, where message order and delivery matter. UDP, by contrast, is connectionless and does not guarantee delivery or order — it trades reliability for lower overhead, which is why it's used for things like live video/audio streaming instead.

3. How This Program Works — Socket Lifecycle

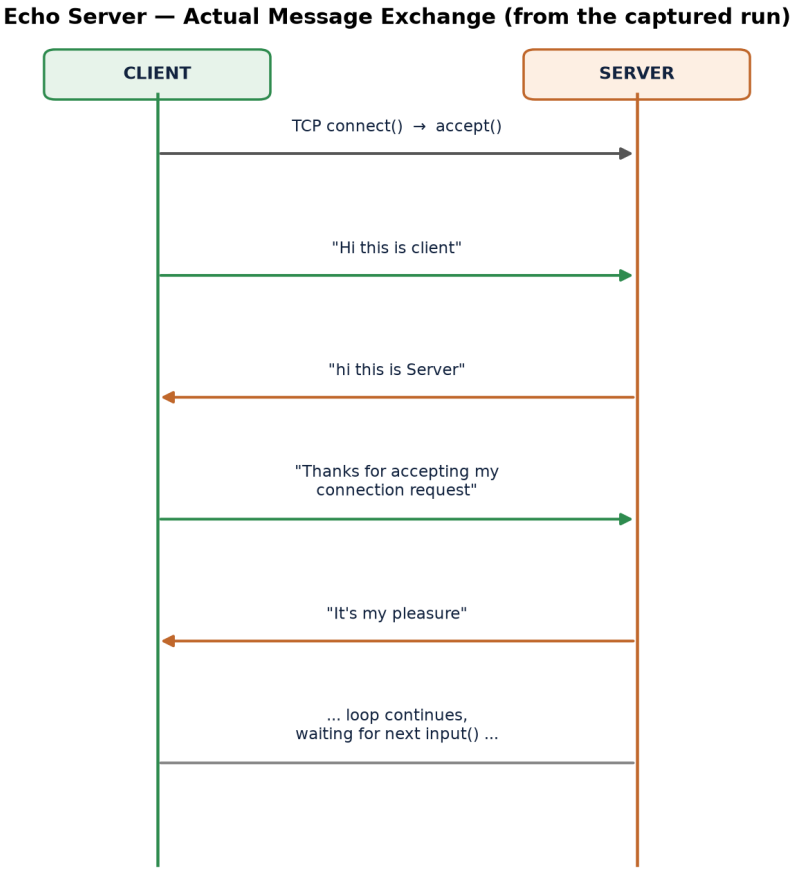
The server and client each follow a fixed sequence of socket calls. The server's job is to wait for a connection; the client's job is to initiate one. The diagram below lines up the exact calls each side makes, in order, and shows where they interact.



Key point: `accept()` *on the server side blocks (pauses execution) until a client actually calls* `connect()`*. This is why you must always start* `server.py` *first and see "Server is waiting for connection..." before starting* `client.py` *— otherwise there's nothing for the client to connect to yet.*

4. Message Flow — What Actually Happened

This isn't a hypothetical flow — it's the literal sequence of messages exchanged when `server.py` and `client.py` were run against each other for the assignment's OUTPUT section:



5. SERVER.PY — Fully Commented

Below is the exact submitted code with an explanatory comment added above every meaningful line, so the purpose of each socket call is clear without needing to cross-reference anything else.

```
import socket
# socket module = Python's wrapper around the OS networking API

# Step 1: create a TCP socket
# AF_INET -> use IPv4 addressing
# SOCK_STREAM -> use TCP (reliable, ordered, connection-oriented)
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Step 2: bind the socket to an address so clients know where to find it
# "127.0.0.1" = localhost (this machine), 5000 = the port number
server.bind(("127.0.0.1", 5000))

# Step 3: start listening for incoming connections
# 5 = max number of connection requests that can queue up while busy
server.listen(5)
print("Server is waiting for connection...")

# Step 4: accept() BLOCKS here until a client connects
# conn -> a brand new socket dedicated to talking to THIS client
```

```
# addr -> (client_ip, client_port) tuple, used only for logging
conn, addr = server.accept()
print("Connected by : ", addr)

# Step 5: main conversation loop
while True:

# receive up to 1024 bytes, then decode raw bytes into a Python string
data = conn.recv(1024).decode()

print("Client Says : ", data)

# exit condition: if the client typed "bye" (any case/spacing), stop
if data.strip().lower() == "bye":
print("Client ended chat")
break

# prompt the person running the server for a reply
string = input("Enter your message to client : ")

# encode the string back into bytes and send it down the socket
conn.send(string.encode())


# Step 6: release both sockets - the per-client one and the listening one
conn.close()
server.close()
```

6. CLIENT.PY — Fully Commented

```
import socket

# Step 1: create a TCP socket, same family/type as the server must use
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Step 2: actively request a connection to the server's address
# This is the call that triggers the TCP handshake with server.accept()
client.connect(("127.0.0.1", 5000))

print("Connected to server")

# Step 3: main conversation loop
while True:

# prompt the person running the client for a message to send
string = input("Enter your message to server : ")
client.send(string.encode())

# wait for and read the server's reply (up to 1024 bytes), decode it
data = client.recv(1024).decode()
print("Server says : ", data)
```

```
# exit condition: if the SERVER'S reply was "bye", stop the loop
if data.strip().lower() == "bye":
print("Client ended chat")
break


# Step 4: release the socket
client.close()
```

7. Key Socket API Reference

Function	Used By	What It Does
`socket()`	Both	Creates a new socket object. AF_INET = IPv4, SOCK_STREAM = TCP.
`bind()`	Server only	Attaches the socket to a specific (IP, port) address so it can be found.
`listen(n)`	Server only	Puts the socket into a passive listening state; n = max queued connections.
`accept()`	Server only	Blocks until a client connects; returns a new socket for that client plus its address.
`connect()`	Client only	Actively opens a connection to a server's (IP, port). Triggers the TCP handshake.
`send()`	Both	Sends bytes over the connection. Strings must be encoded to bytes first (.encode()).
`recv(bufsize)`	Both	Reads up to bufsize bytes from the connection. Blocks until data arrives.
`encode()` / `decode()`	Both	Convert between Python str (text) and bytes (what sockets actually transmit).
`close()`	Both	Releases the socket and frees its resources; ends that side of the connection.

8. Likely Exam / Viva Questions

Q1. Why does the server call bind() but the client doesn't?

The server must advertise a fixed, known address so clients can find it. The client only needs to know the server's address to connect to (via connect()) — the OS automatically assigns the client an ephemeral (temporary) port behind the scenes.

Q2. What happens if you start client.py before server.py?

The connect() call fails immediately with a connection-refused error, because there is no socket listening on 127.0.0.1:5000 yet for the OS to connect to.

Q3. Why do we call .encode() before send() and .decode() after recv()?

Sockets transmit raw bytes, not Python strings. encode() converts a str to bytes (UTF-8 by default) before sending; decode() converts the bytes received back into a readable str.

Q4. What does the second argument to listen() mean?

It sets the backlog — the maximum number of pending connections the OS will queue while the server is busy handling a previous accept(). It does not limit total connections over time.

Q5. Why does accept() return two values, conn and addr?

conn is a NEW socket dedicated to this specific client (the original server socket keeps listening for others); addr is the (IP, port) tuple identifying who connected, used for logging.

Q6. Is this Echo Server concurrent (can it handle multiple clients at once)?

No. accept() is only called once, so this server handles exactly one client, then exits its loop when that client disconnects. Supporting multiple clients would require either a loop around accept() plus threads/processes per client, or an async/select-based event loop.

Q7. What's the difference between server.close() and conn.close()?

conn.close() ends the connection to that one client. server.close() shuts down the original listening socket itself, after which the server can no longer accept new connections at all.

9. Summary — Key Takeaways

- A socket is a network endpoint identified by an (IP address, port) pair.
- TCP (SOCK_STREAM) gives a reliable, ordered, connection-oriented byte stream — the right choice when message delivery and order matter.
- The server's sequence is fixed: socket -> bind -> listen -> accept -> recv/send loop -> close.
- The client's sequence is fixed: socket -> connect -> send/recv loop -> close.
- accept() and connect() are the two calls that actually perform the TCP handshake and pair the client to the server.
- All data crossing a socket is bytes — always encode() strings before sending and decode() bytes after receiving.
- This particular server handles exactly one client per run since accept() is called only once.